SMS Admin

PROJECT TECHNICAL REPORT

Student Management System

A full-stack web application for administrators to manage student records, courses, attendance, and academic information through a secure and user-friendly interface.

DOCUMENT TYPE

Technical Project Report

VERSION

1.0.0

DATE

August 16, 2026

APPLICATION TYPE

Full-Stack Web Application

Table of Contents

1. Executive Summary

The Student Management System (SMS) is a full-stack web application designed to streamline the administrative workflows of educational institutions. It provides administrators with a centralized platform to manage student records, course offerings, student enrollments, and attendance tracking — all through a secure, responsive, and intuitive interface.

Built on a modern technology stack comprising React 18, TypeScript, Vite, and Supabase (PostgreSQL), the system delivers a production-ready single-page application with real-time data operations, role-based access control, and a comprehensive user experience optimized for both desktop and mobile devices.

APPLICATION TYPE

Single-Page Web Application

TARGET USERS

Educational Administrators

AUTHENTICATION

Email / Password (Supabase Auth)

DATABASE

PostgreSQL via Supabase

Key Achievements

- Complete CRUD operations for students, courses, enrollments, and attendance
- Row-Level Security (RLS) enforced at the database level for data isolation
- Responsive design with light/dark mode support across all breakpoints
- Type-safe codebase with zero TypeScript compilation errors
- Production build verified and optimized for deployment

2. Project Overview

The Student Management System addresses the need for a digital, paperless approach to managing academic records. Traditional methods involving spreadsheets and physical files are error-prone, difficult to search, and lack proper access controls. This system provides a structured, searchable, and secure alternative.

Objectives

- **Centralized Data Management:** All student, course, and attendance data stored in a single relational database with referential integrity
- **Secure Access:** Authentication required for all operations; data isolated per administrator account
- **Efficient Operations:** Search and filter capabilities to quickly locate records among potentially hundreds of entries
- **Comprehensive Tracking:** End-to-end academic lifecycle from enrollment through attendance and course completion
- **User-Friendly Interface:** Intuitive design requiring minimal training, accessible from any device

Scope

The system covers four primary functional areas: student record management, course management, enrollment management, and attendance tracking. A dashboard provides an aggregated overview of institutional data. The system supports a multi-admin model where each administrator manages their own set of records independently.

3. Technology Stack

The application is built using industry-standard, well-supported technologies chosen for their reliability, developer experience, and long-term maintainability.

Layer	Technology	Version
Frontend Framework	React	18.3.1
Language	TypeScript	5.5.3

Layer	Technology	Version
Build Tool	Vite	5.4.2
Styling	Tailwind CSS	3.4.1
Icon Library	Lucide React	0.446.0
Client Routing	React Router DOM	6.30.4
Backend / BaaS	Supabase	2.57.4 (client SDK)
Database	PostgreSQL	Managed by Supabase
Authentication	Supabase Auth	Integrated
Code Linting	ESLint + TypeScript ESLint	9.9.1 / 8.3.0
Package Manager	npm	—

Technology Justification

React + TypeScript

React's component-based architecture enables reusable UI elements and efficient rendering through its virtual DOM. TypeScript adds static type checking, catching errors at compile time and improving code maintainability through explicit interfaces and type definitions.

Vite

Vite provides near-instantaneous cold server start and lightning-fast hot module replacement (HMR) during development. Its optimized production build uses Rollup under the hood, producing smaller, more efficient bundles.

Supabase

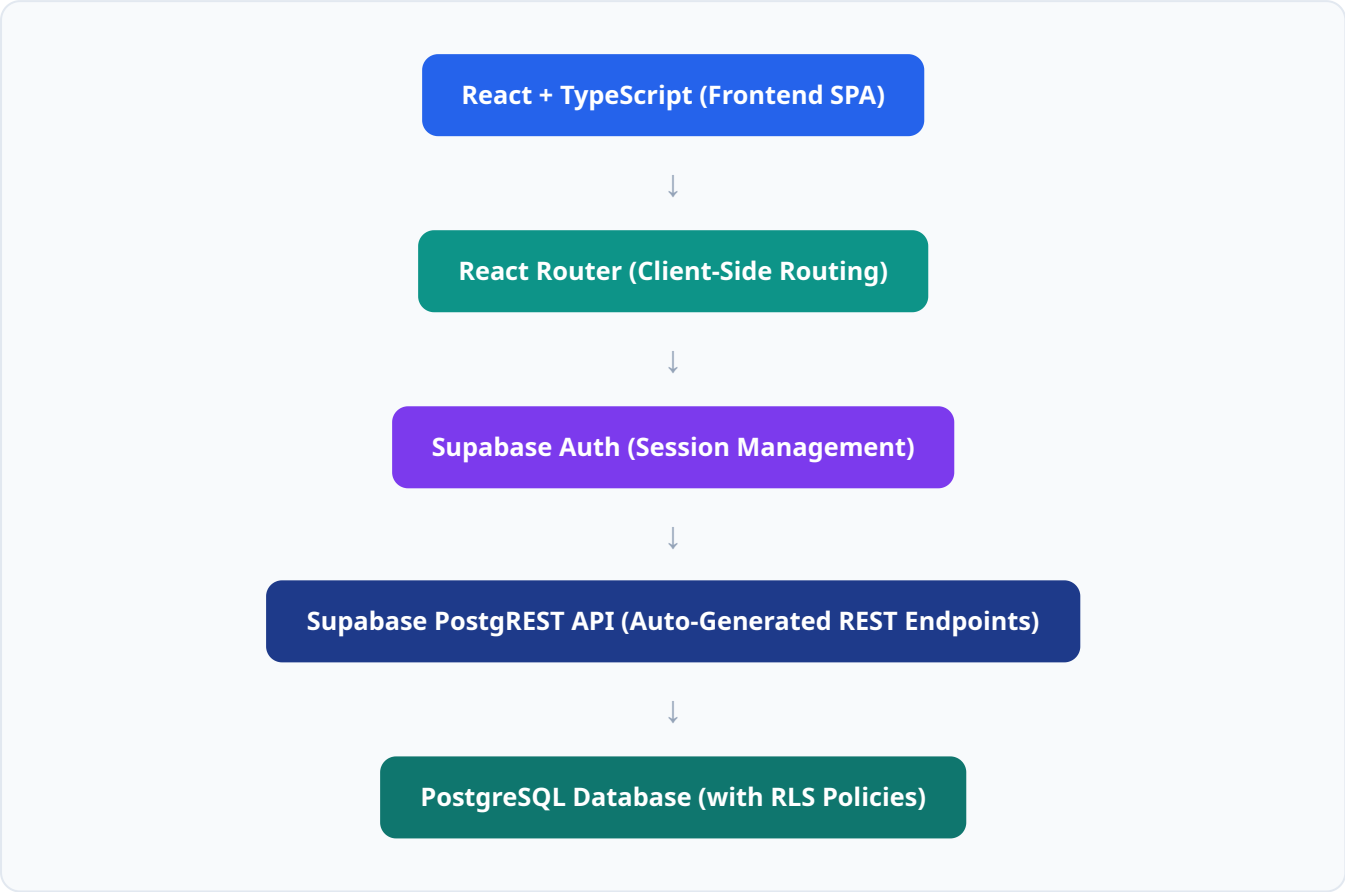
Supabase offers an integrated backend solution combining PostgreSQL, authentication, real-time subscriptions, and storage. Its PostgREST API allows the frontend to communicate directly with the database through a typed client library, eliminating the need for a custom REST API layer while maintaining security through Row Level Security policies.

Tailwind CSS

Tailwind's utility-first approach enables rapid UI development with consistent spacing, typography, and color systems. Its JIT compiler produces minimal CSS containing only the styles actually used in the application.

4. System Architecture

The application follows a client-server architecture where the browser-based frontend communicates directly with the Supabase backend. Supabase's PostgREST layer automatically exposes the PostgreSQL database as a RESTful API, and Row Level Security policies enforce access control at the database level, ensuring security even without a traditional middleware server.



Architectural Patterns

Single-Page Application (SPA)

The application operates as a single-page application with client-side routing. Only one HTML document is loaded; subsequent navigation is handled by React Router without full page reloads, resulting in a fast, app-like user experience.

Context-Based State Management

Global state is managed through React Context API with three dedicated providers:

- **AuthProvider** — Manages the Supabase authentication session, user object, and sign-in/sign-up/sign-out operations
- **ThemeProvider** — Manages the light/dark mode preference with localStorage persistence and system preference detection
- **ToastProvider** — Manages transient notification messages (success, error, info) with auto-dismiss

Direct-to-Database Communication

The frontend uses the Supabase JavaScript client to communicate directly with the PostgreSQL database. All queries are subject to Row Level Security policies, meaning the database itself enforces authorization. This eliminates the need for a custom API server while maintaining strict security boundaries.

Protected Route Pattern

All application routes are wrapped in a protected route guard. When a user is not authenticated, they are redirected to the authentication page. Once authenticated, they gain access to the dashboard, students, courses, and attendance modules.

Data Flow

1. User signs in via Supabase Auth; a JWT session token is issued and persisted
2. Authenticated requests include the JWT, which Supabase uses to identify the user via `auth.uid()`
3. RLS policies evaluate each query against `auth.uid() = user_id`, allowing access only to owned records
4. Query results are returned as typed JSON to the frontend and rendered in React components

5. Feature Specifications

5.1 Authentication & Authorization

Authentication

- Email and password sign-up
- Email and password sign-in
- Session persistence across page reloads
- Automatic token refresh
- Secure sign-out with session cleanup

Authorization

- Protected route guard
- Unauthenticated users redirected to login
- Row-Level Security per administrator
- Data isolation between admin accounts
- JWT-based identity verification

5.2 Dashboard

The dashboard serves as the landing page after authentication, providing an at-a-glance overview of institutional data.

- **Statistical Cards:** Total students, active students, total courses, active courses, total enrollments, and today's attendance count
- **Quick Actions:** Shortcut buttons to add students, add courses, or mark attendance
- **Recently Added Students:** Five most recently created student records with avatars and status badges
- **Recent Attendance:** Five most recent attendance entries with student name, course, and status
- **Attendance Rate:** Percentage of present students for the current day

5.3 Student Management (CRUD)

Capability	Description
Create	Add new student with full profile information via modal form
Read	View all students in a responsive table (desktop) or card layout (mobile)
Update	Edit any student field through the same modal form, pre-populated with existing data
Delete	Remove a student with confirmation dialog; cascading deletes remove related enrollments and attendance
Search	Real-time search by student name or email address
Filter	Filter by status: active, inactive, graduated, or suspended

Student Fields: First name, last name, email (unique), phone, date of birth, gender, address, enrollment date, and status.

5.4 Course Management (CRUD)

Capability	Description
Create	Add new course with code, title, credits, instructor, and capacity
Read	View courses as cards showing code, title, credits, instructor, and enrollment count vs. capacity
Update	Edit any course field through a modal form
Delete	Remove a course with confirmation; cascading deletes remove related enrollments and attendance
Search	Search by course code, title, or instructor name
Filter	Filter by status: active or archived

Course Fields: Code (unique), title, description, credits, instructor, capacity, and status.

5.5 Enrollment Management

- Enroll active students into courses via a dedicated enrollment modal
- View all enrolled students for a specific course
- Remove (unenroll) students from a course
- Prevents duplicate enrollments through a unique database constraint
- Real-time capacity tracking with "Full" indicator when enrollment reaches capacity

- Only active students are available for enrollment

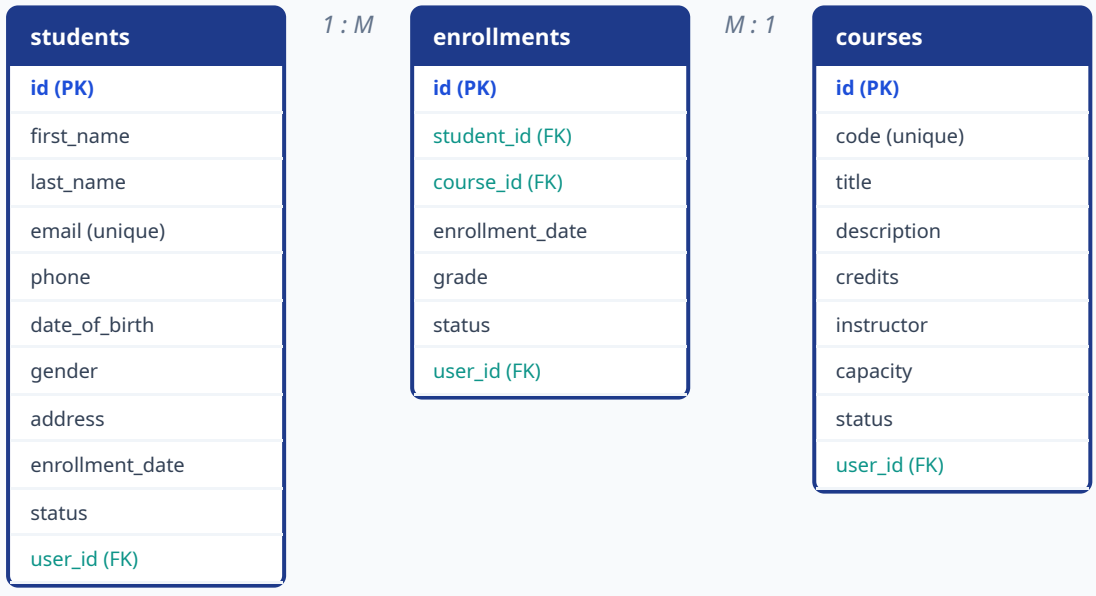
5.6 Attendance Tracking

- Select a course and date to mark attendance for all active students
- Four attendance statuses: Present, Absent, Late, and Excused
- Bulk action: "Mark all present" for efficient entry
- Individual status selection per student with visual highlighting
- Bulk save of all attendance marks in a single operation
- Attendance history with filtering by course, status, and student name
- Delete individual attendance records
- Unique constraint prevents duplicate attendance for the same student/course/date

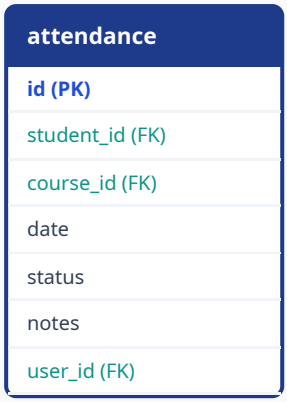
6. Database Schema

The database consists of four relational tables managed by PostgreSQL through Supabase. All tables use UUID primary keys generated by the `pgcrypto` extension. Foreign key constraints with cascade deletes ensure referential integrity.

6.1 Entity Relationship Overview



students 1:M attendance M:1 courses



6.2 Table: students

Column	Type	Constraints	Description
id	uuid	PK, Default gen_random_uuid()	Unique identifier for each student
first_name	text	NOT NULL	Student's first name
last_name	text	NOT NULL	Student's last name
email	text	NOT NULL, UNIQUE	Student's email address (unique across all records)
phone	text	*nullable*	Contact phone number
date_of_birth	date	*nullable*	Date of birth
gender	text	Default 'other'	Gender: male, female, or other
address	text	*nullable*	Home address
enrollment_date	date	NOT NULL, Default CURRENT_DATE	Date the student was enrolled
status	text	NOT NULL, Default 'active'	Status: active, inactive, graduated, or suspended
user_id	uuid	NOT NULL, FK → auth.users, Default auth.uid()	Owner (admin who created the record)
created_at	timestamptz	Default now()	Record creation timestamp
updated_at	timestamptz	Default now(), auto-updated	Last modification timestamp (trigger)

6.3 Table: courses

Column	Type	Constraints	Description
id	uuid	PK, Default gen_random_uuid()	Unique identifier for each course
code	text	NOT NULL, UNIQUE	Course code (e.g., CS101)
title	text	NOT NULL	Course title
description	text	*nullable*	Course description
credits	integer	NOT NULL, Default 3	Credit hours
instructor	text	*nullable*	Instructor name
capacity	integer	Default 30	Maximum student capacity
status	text	NOT NULL, Default 'active'	Status: active or archived
user_id	uuid	NOT NULL, FK → auth.users, Default auth.uid()	Owner (admin who created the record)
created_at	timestamptz	Default now()	Record creation timestamp
updated_at	timestamptz	Default now(), auto-updated	Last modification timestamp (trigger)

6.4 Table: enrollments

Column	Type	Constraints	Description
id	uuid	PK, Default gen_random_uuid()	Unique identifier for each enrollment
student_id	uuid	NOT NULL, FK → students (CASCADE)	Reference to the enrolled student
course_id	uuid	NOT NULL, FK → courses (CASCADE)	Reference to the course
enrollment_date	date	Default CURRENT_DATE	Date of enrollment
grade	text	*nullable*	Letter grade (A, B, C, D, F) or null if ungraded
status	text	NOT NULL, Default 'enrolled'	Status: enrolled, completed, or dropped
user_id	uuid	NOT NULL, FK → auth.users, Default auth.uid()	Owner
created_at	timestamptz	Default now()	Record creation timestamp

> **Unique Constraint:** `UNIQUE(student_id, course_id)` — prevents a student from being enrolled in the same course more than once.

6.5 Table: attendance

Column	Type	Constraints	Description
id	uuid	PK, Default gen_random_uuid()	Unique identifier for each attendance record
student_id	uuid	NOT NULL, FK → students (CASCADE)	Reference to the student
course_id	uuid	NOT NULL, FK → courses (CASCADE)	Reference to the course
date	date	NOT NULL	Date of the attendance record
status	text	NOT NULL	Status: present, absent, late, or excused
notes	text	*nullable*	Optional notes about the attendance
user_id	uuid	NOT NULL, FK → auth.users, Default auth.uid()	Owner
created_at	timestamptz	Default now()	Record creation timestamp

> **Unique Constraint:** `UNIQUE(student_id, course_id, date)` — prevents duplicate attendance records for the same student, course, and date.

6.6 Database Indexes

Indexes are created on frequently queried columns to optimize read performance:

Table	Index Name	Column(s)
students	idx_students_status	status
students	idx_students_user_id	user_id
courses	idx_courses_status	status
courses	idx_courses_user_id	user_id
enrollments	idx_enrollments_student_id	student_id
enrollments	idx_enrollments_course_id	course_id
enrollments	idx_enrollments_user_id	user_id
enrollments	idx_enrollments_status	status
attendance	idx_attendance_student_id	student_id

Table	Index Name	Column(s)
attendance	idx_attendance_course_id	course_id
attendance	idx_attendance_date	date
attendance	idx_attendance_user_id	user_id
attendance	idx_attendance_status	status

6.7 Database Triggers

A PL/pgSQL trigger function `update_updated_at_column()` automatically updates the `updated_at` column whenever a record is modified in the `students` or `courses` tables. This ensures accurate tracking of when records were last modified without relying on application logic.

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Applied to: students, courses
-- Fires: BEFORE UPDATE, FOR EACH ROW
```

7. Security Implementation

Security is implemented at multiple layers, with the database serving as the authoritative enforcement point through PostgreSQL's Row Level Security (RLS) feature.

7.1 Authentication Layer

- Powered by Supabase Auth using industry-standard JWT (JSON Web Token) sessions
- Email and password authentication with Supabase's built-in credential hashing
- Sessions persisted in browser local storage with automatic token refresh
- Sign-out clears the session and revokes access
- Email confirmation is disabled by design for streamlined admin onboarding

7.2 Row Level Security (RLS)

RLS is enabled on all four database tables. Once enabled, tables are locked down and inaccessible until explicit policies are defined. This means that even if the anon key is compromised, no data can be accessed without a valid authenticated session.

> **Security Design Principle:** Each table has exactly 4 separate policies (one per CRUD verb: SELECT, INSERT, UPDATE, DELETE), never a combined FOR ALL policy. This granular control ensures that access can be precisely tuned per operation.

Policy Structure

All policies follow the same ownership-based pattern:

```
-- SELECT policy: Read access check
CREATE POLICY "select_own_students" ON students
  FOR SELECT TO authenticated
  USING (auth.uid() = user_id);

-- INSERT policy: Write access check
CREATE POLICY "insert_own_students" ON students
  FOR INSERT TO authenticated
  WITH CHECK (auth.uid() = user_id);

-- UPDATE policy: Both read and write check
CREATE POLICY "update_own_students" ON students
  FOR UPDATE TO authenticated
  USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);

-- DELETE policy: Read access check for deletion
CREATE POLICY "delete_own_students" ON students
  FOR DELETE TO authenticated
  USING (auth.uid() = user_id);
```

Policy Summary Table

Table	Policy Count	Scope	Ownership Check
students	4	authenticated	auth.uid() = user_id
courses	4	authenticated	auth.uid() = user_id
enrollments	4	authenticated	auth.uid() = user_id
attendance	4	authenticated	auth.uid() = user_id

7.3 Data Isolation

The `user_id` column on every table defaults to `auth.uid()`, meaning new records are automatically associated with the authenticated admin who created them — the frontend does not need to explicitly pass this value. RLS policies then ensure that each admin can only read, modify, or delete their own records. This creates complete data isolation between administrator accounts.

7.4 Cascading Deletes

Foreign key constraints use `ON DELETE CASCADE` to maintain referential integrity. When a student or course is deleted, all related enrollment and attendance records are automatically removed, preventing orphaned data.

7.5 Frontend Security Measures

- **Protected Routes:** All application routes are guarded; unauthenticated users are redirected to the login page
- **Confirmation Dialogs:** Destructive operations (delete) require explicit user confirmation
- **Error Handling:** All database operations include error handling with user-visible feedback via toast notifications
- **No Sensitive Data Exposure:** Environment variables are prefixed with `VITE_` and the service role key is never exposed to the client

8. API Documentation

The application communicates with Supabase's auto-generated PostgREST API directly from the browser. No custom REST endpoints are required. The Supabase JavaScript client provides a typed, promise-based interface for all database operations. All API calls are subject to Row Level Security policies enforced at the database level.

8.1 Authentication API

Operation	Method Call	Description
Sign Up	`supabase.auth.signUp({ email, password })`	Creates a new administrator account
Sign In	`supabase.auth.signInWithPassword({ email, password })`	Authenticates an existing user
Sign Out	`supabase.auth.signOut()`	Ends the current session
Get Session	`supabase.auth.getSession()`	Retrieves the current session state
Auth State Change	`supabase.auth.onAuthStateChange(callback)`	Subscribes to authentication state changes

8.2 Students API

Operation	Method Call	Notes
List All	`supabase.from('students').select('*')`	Ordered by created_at descending

Operation	Method Call	Notes
Create	`supabase.from('students').insert({ ...fields })`	user_id auto-populated by default
Update	`supabase.from('students').update({ ...fields }).eq('id', id)`	updated_at auto-set by trigger
Delete	`supabase.from('students').delete().eq('id', id)`	Cascades to enrollments & attendance

8.3 Courses API

Operation	Method Call	Notes
List All	`supabase.from('courses').select('*')`	Ordered by created_at descending
Create	`supabase.from('courses').insert({ ...fields })`	user_id auto-populated by default
Update	`supabase.from('courses').update({ ...fields }).eq('id', id)`	updated_at auto-set by trigger
Delete	`supabase.from('courses').delete().eq('id', id)`	Cascades to enrollments & attendance

8.4 Enrollments API

Operation	Method Call	Notes
List by Course	`supabase.from('enrollments').select('*, student:students(id, first_name, last_name, email)').eq('course_id', id)`	Includes joined student data
Count All	`supabase.from('enrollments').select('id', { count: 'exact', head: true })`	Head request for count only
Create	`supabase.from('enrollments').insert({ student_id, course_id })`	Unique constraint prevents duplicates
Delete	`supabase.from('enrollments').delete().eq('id', id)`	Removes (unenrolls) a student

8.5 Attendance API

Operation	Method Call	Notes
List History	`supabase.from('attendance').select('*, student:students(...), course:courses(...)').order('date', { ascending: false }).limit(100)`	Includes joined student & course data
List by Date	`supabase.from('attendance').select('*').eq('course_id', id).eq('date', date)`	For loading existing marks
Bulk Create	`supabase.from('attendance').insert([{ student_id, course_id, date, status }, ...])`	Bulk insert for all students
Delete by Course+Date	`supabase.from('attendance').delete().eq('course_id', id).eq('date', date)`	Clears existing marks before re-save
Delete Single	`supabase.from('attendance').delete().eq('id', id)`	Removes individual record

8.6 Client Configuration

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: {
    persistSession: true,
    autoRefreshToken: true,
    detectSessionInUrl: true,
  },
});
```

9. User Interface & Experience

9.1 Design System

Element	Specification
Typography	Inter font family (Google Fonts) with weights 400, 500, 600, 700
Line Height	1.5 for body text, 1.2 for headings
Spacing System	8px base unit with consistent application throughout
Color System	Six color ramps: primary (blue), secondary (teal), accent (amber), success (green), warning (orange), error (red), plus neutral grays

Element	Specification
Border Radius	8px for cards, 6px for inputs, 4px for badges
Shadows	Subtle shadow-sm for cards, shadow-lg for modals and toasts
Dark Mode	Full dark mode support via Tailwind's class strategy with system preference detection

9.2 Responsive Breakpoints

Breakpoint	Width	Layout Adaptation
Mobile	< 640px	Single-column layout, card-based lists, collapsible sidebar with hamburger menu
Tablet (sm)	≥ 640px	Two-column grids for stat cards and forms
Desktop (lg)	≥ 1024px	Fixed sidebar, multi-column grids, table-based data views

9.3 Interactive Components

Modals

- Backdrop with blur effect
- Escape key to close
- Click outside to dismiss
- Body scroll lock when open
- Scale-in animation

Toast Notifications

- Success, error, and info variants
- Auto-dismiss after 4 seconds
- Manual close button
- Slide-up animation
- Stacked positioning

Confirmation Dialogs

- Warning icon for destructive actions
- Contextual messages
- Loading state during operation
- Cancel and confirm buttons

Status Badges

- Color-coded by status type
- Student, course, enrollment, attendance
- Light and dark mode variants
- Capitalized labels

9.4 Animations

- **fade-in:** Opacity transition for overlay layers
- **slide-up:** Combined opacity and Y-axis translation for toasts and mobile sidebar
- **scale-in:** Combined opacity and scale for modal dialogs
- **Hover states:** Subtle background and shadow transitions on interactive elements

10. Project Structure

The codebase is organized by cohesion, following the single responsibility principle. Each directory serves a clear purpose, and files are named to reflect their content.

```
project-root/
├── index.html                 HTML entry point with font preconnects
├── package.json               Dependencies and build scripts
├── vite.config.ts             Vite build configuration with path alias
├── tailwind.config.js         Tailwind theme with custom color ramps
├── postcss.config.js          PostCSS configuration
├── tsconfig.app.json          TypeScript compiler options
├── eslint.config.js           ESLint configuration
├── README.md                  Project documentation
├── .env                       Environment variables (Supabase credentials)
└── src/
    ├── main.tsx               React entry point
    ├── App.tsx                Root component with routing & providers
    ├── index.css              Global styles, Tailwind directives, components
    ├── vite-env.d.ts          Vite client type declarations
    ├── types/
    │   └── index.ts          All TypeScript interfaces & type aliases
    ├── lib/
    │   ├── supabase.ts       Supabase client singleton
    │   └── format.ts         Date formatting & utility functions
    ├── contexts/
    │   ├── AuthContext.tsx    Authentication state & operations
    │   └── ThemeContext.tsx   Light/dark mode management
    ├── components/
    │   ├── Layout.tsx         Sidebar navigation & header shell
    │   ├── Modal.tsx          Reusable modal dialog
    │   ├── ConfirmDialog.tsx  Confirmation dialog for destructive actions
    │   ├── StatusBadge.tsx    Color-coded status indicators
    │   ├── EmptyState.tsx     Placeholder for empty data views
    │   ├── Loader.tsx         Loading spinner & full-page loader
    │   └── Toast.tsx          Toast notification system & provider
    └── pages/
        ├── AuthPage.tsx       Sign-in / sign-up screen
        ├── Dashboard.tsx      Overview with stats & recent activity
        ├── StudentsPage.tsx   Student CRUD with search & filter
        ├── CoursesPage.tsx    Course CRUD with enrollment management
        └── AttendancePage.tsx Attendance marking & history
```

File Count Summary

Category	File Count	Purpose
Pages	5	One per major application route
Components	7	Reusable UI building blocks
Contexts	2	Global state providers (auth, theme)

Category	File Count	Purpose
Libraries	2	Supabase client & formatting utilities
Types	1	Shared TypeScript type definitions
Core Files	3	App.tsx, main.tsx, index.css
Total Source Files	**20**	

11. Setup & Deployment

11.1 Prerequisites

- Node.js version 18 or higher
- npm (Node Package Manager)
- A Supabase project with URL and anon key

11.2 Environment Configuration

The application requires two environment variables defined in a `.env` file at the project root:

```
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-public-key
```

> **Security Note:** The anon key is safe to expose in client-side code because data access is controlled by Row Level Security policies. The service role key must never be included in client-side code.

11.3 Build Commands

Command	Script	Description
Install Dependencies	`npm install`	Installs all required packages
Development Server	`npm run dev`	Starts Vite dev server with HMR
Production Build	`npm run build`	Compiles and bundles for production
Preview Build	`npm run preview`	Serves the production build locally
Type Check	`npm run typecheck`	Runs TypeScript compiler in check mode
Lint	`npm run lint`	Runs ESLint on all source files

11.4 Deployment

The production build generates static assets in the `dist/` directory. These assets can be deployed to any static hosting platform (Vercel, Netlify, Cloudflare Pages, AWS S3 + CloudFront, etc.). No server-side runtime is required for the frontend; all dynamic operations are handled by Supabase.

1. Run `npm run build` to produce optimized static assets
2. Deploy the `dist/` directory to your hosting platform
3. Configure environment variables on the hosting platform
4. Set up a custom domain (optional) and enable HTTPS

11.5 Database Migration

The database schema is managed through a single migration file applied via the Supabase migration system. The migration is idempotent, meaning it can be safely re-run without causing errors. It creates all tables, indexes, constraints, RLS policies, and triggers in a single operation.

12. Quality Assurance

12.1 Build Verification

> **Production Build:** PASS — The application compiles successfully with Vite, producing optimized static assets with no errors.

> **TypeScript Type Check:** PASS — The `tsc --noEmit` check completes with zero errors, confirming full type safety across the codebase.

12.2 Code Quality Measures

- Strict TypeScript configuration with `strict: true`
- ESLint with React Hooks and React Refresh plugins
- Explicit type annotations on all function parameters
- No `any` types used in the codebase
- Consistent import style using the `@/` path alias
- Component-level error handling with user-visible feedback

12.3 Security Verification

- Row Level Security verified as enabled on all four tables
- All 16 policies (4 per table) confirmed scoped to `authenticated` role
- Ownership checks using `auth.uid() = user_id` confirmed on all policies
- Cascading delete constraints verified on all foreign keys
- Unique constraints verified for enrollments and attendance

13. Future Enhancements

The following enhancements are recommended for future iterations:

Enhancement	Description
Role-Based Access Control	Multiple roles (admin, teacher, student) with differentiated permissions and dashboards
Student Self-Service Portal	A separate interface where students can view their own attendance and grades
Grade Management	Dedicated grade entry interface with GPA calculation and transcript generation
Report Generation	Export attendance reports, student lists, and course rosters as PDF or CSV
Notifications	Email or in-app notifications for low attendance warnings or enrollment changes
Calendar Integration	Visual calendar view for attendance and course scheduling
Bulk Import/Export	CSV import for bulk student or course creation
Audit Trail	Track all data modifications with timestamps and user attribution

14. Conclusion

The Student Management System delivers a comprehensive, production-ready solution for educational administration. It successfully addresses all specified requirements: authentication and authorization, student CRUD operations, course management, search and filter capabilities, responsive UI, and a dashboard overview.

The system's architecture — combining a React/TypeScript frontend with a Supabase/PostgreSQL backend — provides a robust foundation that is both scalable and maintainable. Security is enforced at the database level through Row Level Security, ensuring data isolation and protection regardless of client behavior. The responsive, accessible user interface with dark mode support delivers a modern experience across all device types.

With zero TypeScript compilation errors, a verified production build, and a well-documented codebase, the project is ready for deployment and real-world use. The modular architecture supports straightforward extension with additional features as institutional needs evolve.

DELIVERABLES STATUS

VERIFICATION STATUS

Build: PASS **Types: PASS** **RLS: Verified**

Source Code README Database Schema API Docs

Student Management System — Project Technical Report — Version 1.0.0 — August 2026

This document was generated as a comprehensive project report covering all aspects of the application.